

PungentFunk Utilities

Architecture and Design Principles Bible

Stable architecture, product design, Unity Foundations alignment, and Unity Editor implementation doctrine separated from volatile script snapshots

This document records durable architecture, UI doctrine, Unity Editor implementation selection rules, package boundaries, interoperability standards, accessibility expectations, and definition-of-done criteria for PungentFunk Utilities. Update this bible when the philosophy, package boundaries, UI doctrine, or interoperability model changes. Update snapshot audits separately whenever scripts change.

Doctrine update note: this version integrates the offline Unity Manual documentation archive at a principle level. The audited material covered the Unity Editor, UI Toolkit, IMGUI migration, custom windows, custom inspectors, property drawers, TreeView/ListView patterns, overlays, EditorTools, Scene View, Gizmos/Handles, Unity Search, shortcuts, AssetDatabase, Package Manager, assembly definitions, prefabs, domain/code reload, workspace/windowing, and UI state/performance. It does not include tutorial-level API steps or code snippets.

1. Scope and North Star

PungentFunk Utilities should be treated as a family of Unity editor/runtime utility assets, not as a single monolithic toolbox. Each lab or suite should be independently releasable, while the larger bundle should expose a cohesive control panel, shared visual language, consistent interaction vocabulary, and optional cross-lab enhancements.

The core promise is simple: automate tedious game-development workflows without adding new tedium. Every tool should be understandable at a glance, safe to try, powerful enough to scale through presets and profiles, and integrated into the Unity Editor surface where the user naturally needs it.

Release model

- Core package: shared registry, lab launcher, theme system, preferences, common editor UX helpers, optional integration contracts, design-system widgets, implementation-selection doctrine, shared scan/result models, and documentation patterns.
- Standalone lab packages: independently useful products such as Colour Lab, Audio Lab, Asset Placement Lab, Texture Lab, Scene Workflow Lab, Asset Preview and Export Lab, Project Audit and Authoring Lab, Content Generation Lab, UI and Feedback Lab, and later Environment, Settings, Action, Agent, Appearance, and Visualization labs.
- Bundle package: installs all labs and unlocks cross-lab linking, shared dashboards, richer context menus, Scene View integrations, graph/canvas handoffs, and coordinated workflows between tools.
- Project adapters: thin game-specific wrappers that pre-fill generic utility settings, preserve old menu paths, or translate project-specific data into generic profiles without polluting generic packages.

Definition of a PungentFunk utility

- It solves a broadly reusable Unity workflow problem.
- It uses generic names, generic data models, stable IDs, and user-configurable presets instead of hardcoded project assumptions.
- It works in a clean project, or clearly declares optional dependencies that can be absent without compile/runtime errors.
- It has preview or dry-run behaviour before destructive changes and records Undo where scene or asset changes are made.
- It uses the shared visual language and persistence helpers unless there is a deliberate, documented reason not to.
- It chooses an appropriate Unity Editor implementation pattern for the workflow rather than forcing every tool into one window, inspector, overlay, graph, or UI technology.

2. Product Architecture

Layer	Purpose	Rules
PungentFunk Utilities Core	Registry, launcher/control panel, lab menu routing, shared theme, preferences, shared widgets, package status metadata, shared scan/result models, design-system tokens, and implementation doctrine.	Every lab may depend on Core. Core must not depend on any lab. Keep Core small, stable, conservative, and boring.
Runtime model assemblies	ScriptableObjects, MonoBehaviours, data models, adapters, and runtime services used outside the Unity Editor.	No UnityEditor references. No editor-only menu logic. Use stable IDs, version fields, runtime-safe defaults, and serialized data designed for migration.
Editor lab assemblies	Editor windows, inspectors, scanners, generators, validators, TreeViews, Scene GUI helpers, overlays, graph tools, and authoring panels.	May depend on UnityEditor and Core. Scan/apply operations must be explicit, cancelable where expensive, and cached. Drawing code should not own mutation logic.
Optional bridges/adapters	Code that connects two labs or a lab to third-party packages.	Must be removable. Use version defines, reflection, asmdef constraints, isolated bridge packages, package metadata, or registry-based service discovery.
Project adapters	Game-specific wrappers and presets outside the generic asset.	Thin only. They may reference game classes; generic packages must never compile against them.

Dependency direction

- Allowed: Lab -> Core, Editor -> Runtime, Project Adapter -> Generic Lab, Bridge -> Lab A + Lab B.
- Discouraged: Lab A directly compiles against Lab B unless Lab B is a declared required dependency for that product.
- Forbidden: Core -> Lab, Runtime -> Editor, Generic Lab -> game-specific project class, and cyclic lab dependencies.
- Preferred optional link: detect availability by registry ID, type lookup, define symbol, optional asmdef or bridge package, package metadata, or service lookup; show a helpful disabled state when absent.

Labs should own their domain models and workflows. Shared abstractions should move to Core only when at least two labs truly need them and they are stable enough to support as public API. Avoid pre-emptively centralising immature helpers.

3. Editor UI and UX Principles

The suite should have a cohesive visual identity, but not a rigid one. Consistency should come from shared theme colours, typography rules, spacing scale, button/status/box styles, icons, interaction conventions, safety patterns, registry metadata, lab navigation, and reusable widgets. Consistency should not mean every workflow is forced into the same vertical inspector layout.

Unity Foundations alignment

PungentFunk tools should feel native to the Unity Editor while retaining a recognisable PungentFunk identity. Unity-native design alignment should influence component semantics, interaction states, authoring flows, windowing, overlays, messaging, iconography, typography, content organisation, accessibility, and keyboard/focus behaviour. The result should be a suite that feels professionally integrated, not a collection of novelty skins.

- Use Unity-standard component meaning first: buttons trigger actions, toggles enable states, dropdowns choose one option, radio groups choose mutually exclusive modes, sliders adjust continuous ranges, numeric fields set exact values, object fields reference assets/objects, search fields filter, tabs separate workflows, toolbars collect compact modes/actions, progress bars report long work, and help boxes explain state.
- Design by authoring context: simple selected-object configuration belongs near the Inspector; dense asset authoring belongs in a dedicated window; spatial manipulation belongs in Scene tools, handles, or overlays; short selected-object actions belong in context menus.
- Use familiar Unity affordances: selection, focus, disabled state, hover, active/on, warning, error, success, override, lock, refresh, apply, search, filter, and drag/drop states should behave predictably across labs.
- Prefer semantic UI over decorative UI: colour, icons, typography, and spacing should orient the user and clarify workflow; they should not compete with the user content.

Responsive layout rules

- The window must remain usable at small sizes. Do not assume a wide monitor.
- Prefer vertical scrollbars over horizontal scrollbars. Avoid horizontal scrolling unless the data is inherently matrix-like, timeline-like, graph-like, or a deliberate canvas workspace.
- Use wrapping rows, compact cards, collapsible sections, tabs, paging, and column stacking before resorting to horizontal scrolling.
- All important shared borders between subpanels should be draggable where panel proportions matter; persist dimensions through `UtilityWindowPrefs`, view data, or equivalent UI Toolkit state.
- Avoid dead space. If a row or card has large blank interior gaps, shrink padding, wrap controls, collapse secondary metadata, or move low-priority content into details.

- Controls should not jump around during refresh. Important actions should not be buried below a large scroll region.

Visual information design

- Use `UtilityWindowTheme` for headers, panels, section titles, count/status pills, tinted buttons, warning/info/help boxes, splitter handles, and shared text styles.
- Use colour intentionally for status, category, severity, selection, or lab identity. Avoid unrelated colour noise and avoid using colour as the only differentiator.
- Use icons, symbols, tags, badges, lock markers, warning markers, count chips, legends, and labels for repeated states.
- Use diagrams, mini graphs, swatch grids, matrices, timelines, heatmaps, or graph canvases only when they clarify relationships faster than text.
- Use concise text in the main panel; put long explanations in foldouts, documentation popups, help panels, or external docs.
- Use typography to establish hierarchy: clear headers, restrained font sizing, readable body text, consistent label/action casing, and no unnecessary visual shouting.

Colour, iconography, and typography doctrine

- Theme colour roles should map to semantic roles such as background, panel surface, input surface, selected state, disabled state, info, warning, error, success, destructive, experimental, and lab identity.
- Custom PungentFunk themes may be expressive, but production utility chrome should remain legible in light and dark Unity themes. Hardcoded colours should be treated as exceptions.
- Icons should condense repeated actions or states into simple symbols, but critical meaning must also be available through labels, tooltips, alt text, or nearby text.
- Toolbar icons, asset icons, preview icons, overlay icons, and scene/gizmo icons serve different purposes; each should be designed for its context and readable at small sizes.
- Most UI text should be functional and concise. Use expressive typography sparingly for lab identity, not as a substitute for clear layout.

4. Window and Panel Layout Design Principles

Each window and panel should be designed deliberately. A utility layout is part of its functionality, not just presentation. The correct layout must be chosen based on workflow, information density, user intent, frequency of use, preview needs, risk level, authoring context, and comparison needs.

Layouts should be purpose-built

- Identify whether the user is configuring settings, scanning assets, comparing results, editing a selected item, previewing visual output, running a batch operation, iterating on generated content, monitoring runtime/debug state, navigating modules, authoring data assets, manipulating scene content, or applying scene/asset changes.
- The layout should support that workflow directly. Avoid defaulting every tool to a long vertical inspector-style panel.
- Inspector-like layouts are useful for simple configuration, but not always appropriate for comparison, preview, generation, search, graph-like relationships, or batch review.
- Group by user task and authoring intent, not by internal code structure.

Template	Best for	Design notes
Simple vertical stack	Small settings, single-purpose utilities, lightweight helpers.	Header -> configuration -> actions -> output/status/help. Use for linear workflows only.
Two-column layout	Source/result, configuration + preview, selection + detail.	Left controls/filter/list; right results/preview/details. Use persistent draggable splitters.
Three-panel layout	Advanced authoring, asset browsers, debugging, lab hubs.	Left navigation/source, centre workspace, right details/actions. Collapse or stack at narrow sizes.
Grid/card layout	Asset sets, swatches, prefab previews, icons, generated names, tool launchers.	Cards wrap vertically; use badges, colour chips, icons, and compact metadata.
Table/matrix layout	Coverage tools, validation, dependency maps, compatibility checks, batch reports.	Sorting/filtering/grouping required. Horizontal scroll is acceptable only when column relationships matter.
Canvas/lab workspace	Generation tools, asset placement, node-like/spatial workflows, preview-first creative tools.	Central work area, side panels, contextual toolbar, overlays, mode controls, collapsible advanced settings.
Wizard/step flow	Setup, migration, export, risky batch operations, multi-stage generation.	Show current step, required inputs, validation, preview/dry-run, and final apply/export.
Dashboard/overview	Control panel, lab home, health summaries, scan summaries.	Summary cards, status pills, warnings, search, recent/pinned tools, quick actions. Orientation and triage, not dense control.

Template	Best for	Design notes
Contextual mini panel / inline toolbar	Component tools, terrain helpers, scene-selection tools, asset-specific quick actions.	Compact controls embedded near the relevant Unity context with an Open Full Tool handoff.

Layout selection principles

- Task frequency: common actions should be closer and more visible than rare actions.
- Risk level: destructive/project-wide actions need preview, confirmation, and visual separation.
- Information density: dense information should be grouped, filtered, summarised, and progressively disclosed.
- Comparison needs: if the user must compare multiple items, avoid hiding them behind separate foldouts.
- Preview needs: if visual output matters, give preview space priority.
- Selection flow: if the user selects an item and edits details, use list/detail or grid/detail patterns.
- Authoring context: if content is selected-scene-object specific, prefer Inspector/Scene View integration; if it is asset or workflow dense, prefer a dedicated window.
- Window size: layouts must degrade gracefully at smaller widths.
- Scanning speed: use alignment, spacing, icons, and colour to help the user understand state quickly.
- Cognitive load: prefer fewer visible decisions at once, with advanced options collapsed or contextual.
- Creative flow: generation/design tools should support iteration without burying regenerate, lock, save, preview, or apply actions.

External design disciplines to draw from

- Professional software design: hierarchy, predictable controls, persistent layout, familiar interaction models.
- Web application design: responsive layouts, cards, dashboards, tabs, sidebars, search/filter patterns, progressive disclosure.
- Graphic design: alignment, spacing, contrast, rhythm, grouping, visual hierarchy, restrained colour usage.
- Infographic design: icons, legends, status symbols, compact summaries, visual explanation of complex relationships.
- Poster/editorial design: strong headers, readable sections, clear calls to action, intentional whitespace.
- Data visualization: matrices, charts, graphs, heatmaps, legends, tooltips, meaningful colour scales.
- Creative software: palettes, inspectors, canvases, toolbars, contextual controls, preview-first workflows.
- Accessibility design: keyboard access, readable contrast, non-colour status indicators, scalable text, clear focus states.

Layout anti-patterns to avoid

- Long undifferentiated walls of fields; excessive nested foldouts; large empty gaps caused by rigid sizing.
- Horizontal scrolling except for justified tables/matrices/canvases; controls that jump around during refresh.
- Important actions hidden below large scroll regions; duplicate buttons doing the same thing.
- Dense text where an icon, swatch, badge, graph, table, or preview would be clearer.
- Tool windows that assume a large monitor; unrelated workflows mixed in one panel without grouping.
- Making every utility look identical at the cost of usability.
- Using a graph, canvas, wizard, overlay, or tabbed layout because it seems advanced rather than because the workflow needs it.

5. Progressive Disclosure, Assistance, Accessibility, and Messaging

Tools should be understandable at a glance, but deep enough for advanced users. Show the core workflow first, keep advanced options collapsed by default, explain advanced options contextually, and never hide essential actions in advanced sections.

Tooltips and contextual help

- Every meaningful field should have a tooltip or help affordance.
- Tooltips should explain what the field controls, when to change it, how it differs from related options, and whether the value is destructive, experimental, expensive, or preview-only.
- Enum fields should provide contextual explanation for the currently selected option. Complex enums may need a small description box under the field.
- Tooltips and labels must support the interaction: toggle labels should describe the enabled state, destructive buttons should state the action, and links should use descriptive text rather than generic wording.

Visual assistance and accessibility

- Use colour chips, icons, graphs, diagrams, mini previews, status pills, legends, and examples where they reduce cognitive load.
- Do not rely on colour alone. Pair colour with icons, labels, patterns, shapes, tooltip text, or row status.
- Maintain readable contrast for text, non-text controls, disabled states, focus states, and overlay content.
- Provide visible focus states and logical keyboard navigation where the underlying Unity UI technology supports it.
- Avoid tiny click targets for frequent actions. If a visible icon is small, its clickable region should still be forgiving.
- Prefer plain-language labels for common operations; use technical terms where they are domain-standard and useful.
- Future runtime UI and Feedback Lab components should treat screen-reader accessibility as a hierarchy of meaningful visible interactive/informative elements, not as a duplicate of every decorative element.
- Accessibility updates and notifications should be meaningful and event-driven; avoid excessive per-frame updates for runtime accessibility metadata.

Empty states

- Use empty states for nothing-to-show conditions: no selection, no scan run yet, no results, no package installed, no generated output, no palette selected, or first-time use.
- An empty state should briefly explain what is missing and offer one clear next action when possible.
- Empty states should not use error styling unless something has actually failed.
- Day-zero states should orient the user and invite action; they should not become documentation dumps.

Errors, warnings, and messaging

- Errors should state the problem, likely cause, and next useful step.
- Warnings should identify risk before the user applies changes, especially for shared materials, prefab assets, scene-wide operations, destructive changes, or irreversible exports.
- Info messages should confirm state without blocking the workflow.
- Help boxes and inline status banners should be preferred for recoverable issues. Modal dialogs should be reserved for destructive, irreversible, or workflow-blocking decisions.
- Messages should be precise, polite, constructive, and located close to the control or result they explain.

6. Unity Editor Implementation Doctrine

PungentFunk tools should choose Unity Editor implementation patterns deliberately. This doctrine is not a mandate to rewrite existing ImGui tools; it is a selection framework for future work and for gradual hardening of existing windows. The implementation should serve the workflow, the scale of the data, the expected product lifetime, and the intended release package.

ImGui, UI Toolkit, and migration policy

- ImGui remains acceptable and useful for existing windows, small utility panels, debug tooling, lightweight inspectors, compatibility wrappers, and internal/editor-only workflows where code-driven rendering is the fastest and clearest path.
- UI Toolkit should be evaluated first for new flagship lab windows, persistent complex workspaces, product-facing Asset Store tools, multi-panel layouts, richer styling systems, component-rich views, retained state, UXML/USS-driven presentation, and tools expected to grow significantly over time.
- Do not rewrite merely for fashion. Existing ImGui tools that are stable, maintainable, performant, and visually cohesive may remain ImGui until there is a concrete product, maintainability, accessibility, or UX reason to migrate.
- Hybrid strategies are acceptable. A lab may keep proven ImGui windows while introducing UI Toolkit for new hub views, persistent dashboards, complex data browsers, overlays, or future major versions.
- Implementation choice should be documented at the tool/lab level for major utilities: why this tool is an EditorWindow, custom inspector, TreeView, overlay, context menu, UI Toolkit panel, graph/canvas tool, or hybrid.

Choosing the correct Unity editor pattern

Pattern	Use when	PungentFunk doctrine
EditorWindow / Lab Window	The workflow has multiple sources, filters, previews, generated results, scan/apply stages, tabs, graphs, or a persistent workspace.	Use for full tools, dashboards, labs, scanners, generators, graph/canvas workspaces, and batch workflows. Keep scan/apply logic outside repaint and cache expensive state.
CustomEditor	A tool belongs directly to a selected component or asset and should be available in its Inspector.	Use for compact validation, object-specific toolbars, Open in Lab handoffs, reset/setup helpers, and focused asset editing. Do not overload inspectors with full lab workflows.
Focused Inspector	The user must inspect one asset/object while selection changes elsewhere.	Use where persistent selected-object editing is required but a full custom lab window would waste space.
PropertyDrawer	A serializable field type or attribute needs reusable presentation wherever it appears.	Use for shared field-level UX: IDs, tags, ranges, weighted entries, palette swatches, references, rule rows, or compact metadata blocks.
Scene GUI / Handles	The user edits spatial data directly in the Scene View.	Use for paths, placement handles, surface alignment, gizmo authoring, environment volumes, sockets, and spatial previews. Keep Scene GUI lightweight.
Scene View Overlay / EditorTool	A scene tool needs persistent, compact, mode-aware controls while the user works in the Scene View.	Use for placement modes, scene navigation, debug visual toggles, path tools, terrain/surface helpers, and environment previews. Keep overlays contextual and non-invasive.
TreeView / Multi-column view	The data is hierarchical, expandable, dense, multi-column, or audit-like.	Use for dependency maps, reference scanners, coverage audits, script indexes, lab registries, result trees, and large validation reports. Preserve state and avoid full rebuilds for large data.
Search provider / Query tool	The workflow needs project-wide discovery, query syntax, indexed assets, or reusable filters.	Use when the user thinks in terms of finding/filtering assets, scene objects, commands, or data records. Avoid inventing a separate search language unless the lab genuinely needs one.
Shortcut / Command	The action is frequent, global, or contextual and benefits from keyboard access.	Support shortcuts for high-frequency actions only. Respect global versus contextual command boundaries and avoid collisions with Unity defaults.
Context Menu	A short action is most useful from a selected asset, component, folder, prefab, material, terrain, or scene object.	Use for context-specific quick actions, not broad navigation. Keep menus concise and route full workflows to the relevant lab.
Editor Toolbar / Command Access	The action is frequent, global, or status-like.	Use sparingly for quick launch, active lab shortcuts, debug toggles, scan current selection, or recent tools. Do not mirror the full menu tree.
Graph / Canvas Tool	The workflow involves relationships, branching logic, dependencies, procedural pipelines, reusable nodes, or spatial organisation.	Use only when the canvas structure makes the workflow clearer than lists/forms. Provide inspector/detail panels, navigation aids, organisation tools, and stable serialization.

UI Toolkit and retained UI doctrine

- Use visual trees, UXML, USS, theme style sheets, and shared style tokens when the tool benefits from persistent structure, reusable templates, flexible styling, designer-readable hierarchy, or long-lived product UI.
- Use data binding for editor UI when it reduces manual event wiring and preserves serialized object workflows, Undo, and prefab override expectations.
- Persist view-only state such as scroll position, selection, expansion, and splitter sizes through view data, preferences, or equivalent persistent state. Do not store temporary editor view state in runtime assets unless it has runtime meaning.
- Use ListView or collection views for large repeating records instead of manually instantiating every row in a ScrollView. Recycle/pool recurring elements carefully and reset state before reuse.
- Use USS and theme tokens rather than inline style sprawl. Class/selector architecture should support maintainability, theming, and performance.
- Use UI Toolkit Debugger or equivalent inspection during development to diagnose hierarchy, style, binding, and layout issues, but do not make debug-only UI part of shipped production chrome.

IMGUI layout rules

- Prefer automatic layout for most editor forms, lists, foldouts, rows, and panels so windows can resize without brittle hand-positioned rectangles.
- Use explicit rectangles only when the tool truly needs a fixed drawing area: custom previews, canvases, graphs, matrices, splitters, minimaps, texture previews, or low-level Scene/viewport GUI regions.
- Use horizontal and vertical grouping deliberately to express columns, rows, toolbars, cards, headers, and compact action strips.
- Treat every scroll view, foldout, splitter, and panel as part of a designed workflow, not just a container for controls.
- Avoid recalculating expensive data during repaint. Rendering code draws the current state; services and scan models produce that state.

Shared compound controls and theme widgets

- Repeated UI patterns should become shared PungentFunk controls instead of being copied into every lab.
- Centralise common controls: search bars, labelled sliders, enum-with-description rows, object rows, splitters, status pills, scan-result rows, warning/info boxes, mini toolbars, preview cards, swatch chips, count badges, lock/refresh/apply controls, dependency notices, empty-state cards, and progress panels.
- A control that appears in two or more labs should be considered for `UtilityWindowTheme`, shared editor widgets, or a lab-agnostic helper.
- A repeated control should carry consistent spacing, tooltip behaviour, disabled/missing state, icon language, safety colouring, keyboard/focus behaviour, and text tone.
- Avoid one-off drawing code for common concepts such as warnings, destructive actions, missing dependencies, selected rows, scan summaries, validation states, or operation progress.

GUIStyle, GUISkin, USS, and UtilityWindowTheme policy

- `UtilityWindowTheme` is the package-level design-system wrapper for PungentFunk ImGui tools.
- Do not scatter ad hoc `GUIStyle` definitions inside windows unless they are temporary, strictly local, or clearly tool-specific.
- Style properties such as padding, margin, word wrapping, clipping, fixed size, stretch behaviour, alignment, and rich text should be treated as layout and communication tools, not decoration only.
- Any future `GUISkin`-style grouping should remain subordinate to the PungentFunk theme vocabulary so installed labs feel related even when their layouts differ.
- UI Toolkit styling should map to the same conceptual theme tokens: headers, panels, section titles, pills, buttons, warnings, splitters, disabled states, selected states, focus states, and lab identity colours.

TreeView, list views, and dense data presentation

- Use `TreeView`, `ListView`, `MultiColumnListView`, `MultiColumnTreeView`, Search tables, or equivalent stateful list/table patterns for large or structured data: audit results, dependency maps, registry views, asset trees, validation matrices, scene object hierarchies, and coverage reports.
- Use flat multi-column lists for flat records with comparable fields. Use hierarchical multi-column tree views for expandable parent/child relationships.
- Preserve selection, expansion, sorting, filtering, scroll position, and row identity across refreshes where possible.
- Use stable unique IDs for rows; avoid regenerating row identity unpredictably after scans.
- Small stable trees may build all rows. Large or frequently changing trees should avoid rebuilding the entire hierarchy on every refresh and should prefer building visible or expanded rows where practical.
- Dense views must still be readable: include grouping, status icons, severity colour, row-level actions, concise summaries, and detail panes instead of overwhelming columns.

Custom inspectors and property drawers

- Custom inspectors should be used for object-specific workflows, validation, setup, and compact actions close to the edited component or asset.
- Use serialized-object workflows for asset/component editing where preserving Undo, prefab overrides, and multi-object editing matters.
- Support multi-object editing where the tool naturally affects multiple selected objects. If multi-object editing is unsafe or ambiguous, disable it clearly and explain why.
- Property drawers should be used for reusable serializable data patterns, not for full workflows that belong in a lab window.
- Inspectors should provide Open in Lab handoffs when the workflow becomes too large for inline Inspector UI.

Scene GUI, Handles, overlays, and spatial authoring

- Use Scene GUI and Handles for direct spatial editing: paths, sockets, placement areas, volumes, anchors, handles, alignment targets, and preview gizmos.
- Use `EditorTools` and overlays for persistent scene modes and compact scene-facing controls that need to remain near the authoring surface.
- Keep Scene GUI focused on spatial manipulation and immediate feedback. Put configuration, scan results, batch actions, and documentation in the associated window or overlay.
- Scene GUI should not perform broad scans, heavy asset lookups, or repeated rebuilds during repaint.
- Spatial editing should respect Undo, selection, snapping, preview/dry-run modes, visibility toggles, isolation context, and Prefab Mode context.

- 2D controls inside the Scene View should be compact, mode-aware, and should not obscure the authoring surface.

Graph, canvas, and node-based tool doctrine

Graph and canvas tools should be reserved for workflows where visible relationships are the primary authoring problem. They should not be used merely to make a tool feel advanced.

- Use graph/canvas tooling when the workflow involves branching logic, dependencies, pipelines, state machines, reusable nodes, procedural generation chains, interaction maps, or spatial organisation.
- Avoid graph UIs for simple lists, settings, single-pass generators, or workflows that are clearer as forms, tables, cards, or wizards.
- A graph workspace should usually include a canvas, selected-element inspector, optional blackboard/resource panel, navigation aid such as a minimap when the graph exceeds the viewport, and compact overlay/toolbar controls for common actions.
- Blackboard-like panels are for reusable variables, definitions, assets, rules, resources, or graph-wide metadata; inspector-like panels are for selected node, edge, group, or subgraph details.
- Comments, placemats, groups, portals, and subgraphs are organisation tools for large graphs, not decoration.
- Graph tools should support selection state, zoom/navigation state, Undo, persistence, recoverable broken references, readable empty/error states, and version-tolerant serialization.

Search, shortcuts, and command access

- Search-driven tools should use familiar query/filter concepts, searchable columns, saved filters, and indexed data where the dataset is large enough to justify it.
- Expose lab-specific search fields for local filtering; consider Unity Search provider-style integration only when results should appear in global editor search or cross-project discovery.
- Shortcut actions should be high-frequency, safe, and easy to describe. Use contextual shortcuts when an action only makes sense within a specific window, tool mode, or Scene View context.
- Do not overload the Editor with shortcuts for rare or destructive actions. Destructive shortcut flows need confirmation or preview.

Workspace, reload, and view-state resilience

- Editor windows should tolerate docking, floating, tabbing, closing/reopening, saved layouts, domain reload, script recompilation, Play Mode transitions, and package reimport.
- Static caches, editor callbacks, and event subscriptions must be resilient to reloads. Avoid duplicate registrations and stale references after compilation or Play Mode changes.
- Use InitializeOnLoad-style startup work sparingly and only for lightweight registration, migration checks, or service setup. Heavy scans should require explicit user action.
- Window state should reconstruct from model data plus persisted view preferences; losing a window should not corrupt assets or leave hidden editor operations running.
- Do not assume the active selection, Scene View, or project state remains stable while a utility is open.

Implementation selection summary

The default question for future work should not be "which script do we put this in?" but "where does the user naturally need this control?" Use an EditorWindow for workflows, CustomEditor for selected-object context, PropertyDrawer for repeated field UX, TreeView or multi-column views for dense hierarchy, Scene GUI/Handles for spatial editing, overlay/toolbar for mode controls, graph/canvas for relationship authoring, context menus for quick actions, Search for discovery/query workflows, shortcuts for frequent commands, and optional bridge packages for cross-lab enhancements.

7. Editor Access and Integration Principles

Menu items are the baseline access method, not the only access method. Tools should appear where the user naturally needs them, while avoiding duplicate clutter.

Access method	Principle
Organised menu paths	Use clean Tools/PungentFunk Utilities/... or Tools/Utilities/... roots. Avoid duplicate entries, old project-specific names, and deep nesting without reason. Legacy aliases should open new generic implementations only.
Utility Control Panel / Lab Hub	Every significant utility should register with the central registry. The hub should support search, lab/module/status filtering, recent/pinned tools, lab descriptions, missing optional dependencies, and context-aware launch actions.
Unity Editor toolbar	Expose compact quick-launch buttons, active lab shortcuts, debug/preview toggles, and status indicators only for frequent workflows. Do not duplicate the full menu tree.
Scene View overlay menu	Scene-related tools should use proper overlay integration where useful: placement, surface alignment, scene navigation, gizmo browsing, debug visual toggles, path authoring, terrain/surface helpers, and environment previews.
Right-click context menus	Use concise context actions for relevant scene objects, components, assets, folders, materials, textures, prefabs, ScriptableObjects, terrains, audio assets, palettes, placement groups, and generated markers.

Access method	Principle
Inspector-embedded toolbars	Use compact toolbar rows, validation/status panels, quick actions, mini previews, dependency messages, and Open in Lab buttons in related components/assets.
Cross-utility embedding	Expose related tools inside other PungentFunk labs through service interfaces, reflection-safe adapters, package detection, optional asmdef references, or bridge packages.
Shortcuts and command palette	Support shortcuts for frequent actions such as opening the control panel, toggling debug tools, scanning selection, regenerating output, validating assets, or applying selected previews.
Selection/context-aware launching	When opened from selected terrain, prefab, palette, audio profile, scene object, or placement group, preload the relevant context and fall back gracefully when context is absent.
Recent tools and quick actions	Support recently opened tools, pinned tools, favourite labs, last active module per lab, resume last session, and related-tool handoffs through shared preferences.

Authoring flow rules

- For scene-only GameObject manipulation, prefer Scene View tools, overlays, handles, and Inspector integration before opening a detached dedicated window.
- For asset-heavy workflows, dense configuration, batch authoring, or simultaneous authoring while selection changes, prefer dedicated windows or focused inspectors.
- Use selection and isolation deliberately. If a tool changes the authoring context, make that state visible in both the source context and the destination UI.
- Context switching should not cause large layout jumps; where possible, change content inside a stable panel location.
- If a tool behaves differently in Prefab Mode, Play Mode, or an isolated scene/context, that state must be visible and safe.

8. Registry, Navigation, and Lab Hubs

The registry is the package-level discovery contract. Every utility should have an ID, display name, lab, module, category, sort order, status, description, tags, menu path, capability flags, implementation role, and optional related utility IDs. Lab hubs should organise tools without removing direct window access.

Field	Purpose
Id	Stable machine-readable identifier. Use kebab-case. Never reuse for a different tool.
DisplayName	User-facing name. Generic, clear, and asset-store friendly.
Lab / Module / Category	Lab is product area; module is subdomain; category is menu/search taxonomy.
Description	One-sentence promise: what the tool does and when to use it.
PackageStatus	Stable, Experimental, In Progress, Deprecated, or Project Adapter.
Implementation role	EditorWindow, CustomEditor, PropertyDrawer, TreeView, MultiColumnView, Scene GUI, Overlay, EditorTool, Context Menu, Search Provider, Shortcut Command, Runtime Service, Bridge, Graph Tool, or hybrid.
RelatedUtilityIds	Complementary tool links. Must not imply a hard dependency unless explicitly declared.
Capability flags	Supports selection, context menu, scene overlay, batch actions, lab hub behaviour, TreeView data, Scene handles, inspector embedding, graph workspace, search provider, shortcuts, accessibility metadata, or package bridge behaviour.

Menu policy

- Primary editor menus should live under Tools/PungentFunk Utilities/... or Tools/Utilities/... consistently for the chosen release line.
- Asset context menus should use Assets/PungentFunk Utilities/... .
- GameObject context menus should use GameObject/PungentFunk Utilities/... .
- CreateAssetMenu paths should use one standard root: PungentFunk Utilities/[Lab]/[Asset Type].
- Legacy aliases are allowed only as compatibility routes into the new generic tool and should be marked internally.
- Menu trees should stay curated. Avoid one-item categories, duplicate aliases, and old project-specific names unless they exist solely as compatibility routes.

9. Optional Dependency and Cross-Utility Integration Principles

PungentFunk Utilities should be modular enough for each lab or suite to ship independently, while also becoming more powerful when multiple labs are installed together. Cross-lab links should enhance a tool when another lab is installed, but must not make the base lab fragile.

Dependency levels

- Required dependencies: minimal dependencies that must exist for the tool to compile and function, such as UnityEngine, UnityEditor for editor-only windows, and Core if the package includes shared registry/theme infrastructure.

- Optional integrations: features that activate when another PungentFunk lab or compatible package is installed. These must not produce compile errors when absent.
- Suggested companions: utilities that enhance a workflow but are not required. Missing companion messages should be helpful, calm, and non-blocking.

Optional integration techniques

- Assembly definition optional references, version defines, define constraints, reflection-safe discovery, interface packages, adapter classes in separate integration folders, registry-based service lookup, ScriptableObject loose references, conditional compilation symbols, and soft dependency metadata in descriptors.
- Cross-lab dependencies should be product decisions, not accidental using directives. If a lab requires another lab, document that as part of the standalone product boundary.

Lab / utility	Required base	Optional enhancement points
Utility Core	None	Discovers installed labs; may show status and missing optional bridge packages.
Debug Lab	Core	Scene Workflow for gizmo sources/navigation; Project Audit for debug coverage reports; UI Feedback for notifications.
Asset Placement Lab	Core; placement runtime models	Colour Lab for palette variants; Texture Lab for masks; Scene Workflow for navigation/handles; Project Audit for validation/repair; Audio Lab for surface metadata.
Scene Workflow Lab	Core	Debug Lab for gizmo diagnostics; Asset Placement for path/placement handoff.
Colour Lab	Core; colour runtime models	Texture Lab for palette-to-texture masks; Asset Placement for material variation; UI Feedback for theme previews.
Texture Lab	Core; texture generation models	Colour Lab for palette sources; Asset Preview/Export for previews and packaging.
Audio Lab	Core; audio runtime models	Project Audit for coverage validation; Asset Placement/Scene Workflow for contextual surface tagging; Settings Lab for mixer/runtime config.
Project Audit and Authoring Lab	Core	Inspects other installed labs through optional scan modules; should not own their runtime models.
UI and Feedback Lab	Core	Input System/TMP optional or guarded; runtime presenters remain layout-agnostic and accessibility-aware.
Visualization / Graph Lab	Core	May provide reusable graph, matrix, heatmap, minimap, and inspector widgets to other labs without owning their domain models.

Dependency tracking

- For each lab/utility, track required PungentFunk systems, optional integrations, external Unity package dependencies, Unity version/package requirements, runtime/editor-only status, standalone/bundle suitability, provided services, and consumed services.
- The dependency map should inform Asset Store package splitting, assembly definition boundaries, menu organisation, documentation, QA testing, and future architecture decisions.

10. Runtime, Editor, Data, and Serialization Rules

Runtime package rules

- Runtime scripts must compile in a player build without UnityEditor.
- Runtime ScriptableObjects should contain data and pure runtime logic only.
- Do not store editor-window state in runtime assets unless it is meaningful to gameplay/runtime use.
- Serialized fields should have tooltips and sane defaults.
- Use stable string IDs or GUID-like keys for anything referenced across assets, adapters, graphs, or package boundaries.
- Runtime UI components should expose accessible labels, roles, values, and state where relevant to the target platform and Unity version.

Editor package rules

- Editor windows should use SerializedObject and SerializedProperty when editing assets with existing references.
- Use Undo.RecordObject / Undo.RegisterCreatedObjectUndo for scene-object changes.
- Use AssetDatabase and EditorUtility.SetDirty safely for asset creation and mutation.
- Prompt before opening/changing scenes, editing prefab assets, or applying destructive project-wide changes.
- Prefer explicit Scan, Preview, Apply Selected, Apply All flows for batch operations.
- Separate rendering/drawing code from scanning, asset mutation, scene mutation, graph mutation, and service logic.

Serialization and prefab doctrine

- Respect Unity serialization constraints. If data must survive domain reload, window reload, asset import, prefab overrides, or package upgrade, store it deliberately in serialized assets or stable editor preferences rather than transient static state.
- Prefab-facing tools must distinguish prefab assets, prefab instances, variants, nested prefabs, Prefab Mode isolation, and in-context prefab editing. Do not silently apply scene-instance decisions to prefab assets.
- When editing prefab instances, preserve override semantics and avoid breaking user-authored variant hierarchies.
- Generated assets should preserve GUID/meta stability where possible and avoid delete/recreate churn when update-in-place is safe.

Migration and compatibility

- Avoid destructive runtime class renames for components already used in scenes or prefabs.
- When replacing a project-specific tool, prefer a new generic base plus legacy wrapper/alias rather than two parallel implementations.
- Migration installers should be temporary unless they provide durable package value.
- Class wrappers and menu aliases should preserve old workflows while directing users into the new generic implementation.
- When migrating IMGUI to UI Toolkit, preserve user-facing workflow and data first; implementation replacement is secondary.

11. Asset, Package, and Import Pipeline Doctrine

The offline Unity Manual reinforces that editor utilities are not only UI; they are also part of Unity asset, package, import, and compilation workflows. PungentFunk tools must respect those systems rather than treating the project as a generic file folder.

Package and distribution doctrine

- Design labs so they can ship as standalone products and as a bundle. Use package boundaries, asmdefs, samples, documentation, and optional bridges deliberately.
- Support Unity package workflows and Asset Store-style asset package workflows where appropriate. The project should not assume one distribution channel forever.
- Keep package content organised: runtime code, editor code, samples, documentation, icons, presets, and migration helpers should have predictable locations.
- Do not rely on project-specific folder paths. Any default output folder should be configurable, safe, and clearly stated.
- Use special Unity folders deliberately: Editor for editor-only scripts, Editor Default Resources for editor-only resources when justified, Gizmos for gizmo icons, Resources only when a runtime lookup genuinely requires it, and StreamingAssets only for raw runtime files.

AssetDatabase and import workflow doctrine

- AssetDatabase is an editor-only capability and should remain isolated to editor assemblies.
- Asset creation, copy, move, rename, import, delete, and save operations should be explicit and previewed when they affect many assets.
- Batch asset operations should be grouped so Unity does not perform unnecessary refresh/import work after each individual file change.
- Avoid triggering domain reloads, asset refresh loops, or import churn unnecessarily. Long asset workflows should report progress and remain cancelable where practical.
- Tools that scan or generate assets should distinguish source assets, generated assets, temporary preview assets, and user-authored assets.
- Generated asset names should be deterministic where useful, collision-safe, and reversible. Never overwrite user-authored assets without clear consent.

Compilation, reload, and startup doctrine

- Assembly definitions should reduce compile scope, clarify dependencies, and support modular product packaging.
- Define constraints, version defines, and optional bridge asmdefs should express product boundaries more cleanly than scattered conditional code.
- Initialize-on-load behavior should be lightweight and deterministic. Registration and metadata setup are appropriate; broad scans, asset writes, and scene mutation are not.
- Domain reload and code reload should be treated as normal editor events. Utilities must recover gracefully, avoid duplicate subscriptions, and rebuild volatile caches only when needed.
- Static state should be treated as a cache, not as source-of-truth user data.

12. Performance and Safety Standards

Performance

- Do not scan all scenes/assets on every OnGUI repaint or visual refresh.
- Cache scan results and rebuild only when settings change or the user requests refresh.
- Use EditorApplication.update queues, delayed calls, or staged work for long-running preview/icon/export operations.
- Use cancelable progress bars or visible progress panels for expensive scans and asset operations.
- Avoid Thread.Sleep in editor tools.
- Avoid SceneView.RepaintAll unless broad repaint is truly necessary. Prefer the active window or active SceneView.
- For large trees/tables/graphs, preserve row or node identity and refresh only the necessary state where practical.
- Runtime accessibility metadata should be created for key visible elements and updated only when meaningful state changes.
- Large UI Toolkit views should minimise visible element count, use list virtualization/pooling where appropriate, and avoid unnecessary style/layout churn.

Safety

- Dry-run or preview first for destructive and batch operations.
- Make dangerous toggles visually obvious and default them to safe behaviour.
- Do not overwrite existing user data unless explicitly enabled.
- Do not modify shared materials, prefab assets, imported assets, or scenes by default without a warning.
- When applying results, show a summary of changed, ignored, and failed items.
- Scene handles, overlays, context menu actions, graph mutations, and inspector quick actions must still respect Undo and confirmation rules where relevant.
- Tools should be clear about whether they operate on selection, active scene, all open scenes, prefab assets, project assets, generated groups, or all matching files.

13. Lab Taxonomy

Lab	Product role	Primary ownership
Utility Core	Shared shell and UX infrastructure.	Registry, control panel, lab menus, tray, theme, preferences, editor performance helpers, status vocabulary, shared scan models, shared widgets, implementation role vocabulary, design-system tokens, and package/distribution doctrine.
Debug Lab	Runtime/editor debug control and signal observation.	Debug channels, router, signal relay, reflected fields/actions, scheduled debug calls, router panel, Scene/debug integration points.
Asset Placement Lab	Reusable scene asset placement and procedural dressing.	Asset sets, rules, scatter/grid/socket/group workflows, placement results, scene application, Scene handles, overlays, spatial previews.
Scene Workflow Lab	Scene authoring speed tools.	Scene navigation, gizmo browsing/sources, surface alignment, path authoring helpers, Scene View assistance.
Colour Lab	Palette creation, analysis, accessibility, and application.	Palette assets, swatches, harmony, contrast, deficiency preview, palette generation and material/UI application.
Texture Lab	Procedural texture/mask generation and array baking.	Texture settings, procedural generator, texture-array sources, bake validation, preview surfaces.
Audio Lab	Surface/contact audio data and coverage validation.	Audio materials, clip sets, interaction profiles/matrices, terrain mappings, contact router/classifier, coverage scanners.
Asset Preview and Export Lab	Preview/render/export support for reusable assets.	Prefab icons, prefab extraction, font previews, output packaging and preview queues.
Project Audit and Authoring Lab	Generic authoring diagnostics and batch tools.	Reference assignment, terrain usage, tuning copy, rename, coverage matrix, token validation, tooltip notes, TreeView-style result browsing.
Content Generation Lab	Reusable text/name generation.	Name lists, MadLib lists, name-generation window, token preview and export flows.
UI and Feedback Lab	Reusable input prompt and UI feedback seed tools.	Input prompt icon tooling now; runtime presenters should remain optional, layout-agnostic, and accessibility-aware.
Visualization / Graph Lab	Reusable visualisation widgets and graph/canvas infrastructure.	Matrix views, heatmaps, graphs, canvas workspaces, minimaps, blackboards/resource panels, selected-element inspectors, graph organisation aids, and reusable data-view widgets.

14. Package Hardening Rules

Namespaces

- Use package namespaces before release. Recommended roots include PungentFunk.Utilities, PungentFunk.Utilities.Runtime, PungentFunk.Utilities.Editor, PungentFunk.Utilities.Editor.Core, PungentFunk.Utilities.Editor.Theme, PungentFunk.Utilities.Placement, PungentFunk.Utilities.Audio, PungentFunk.Utilities.Colour, PungentFunk.Utilities.Generation, PungentFunk.Utilities.SceneTools, PungentFunk.Utilities.UI, and PungentFunk.Utilities.Visualization.
- Do not destructively rename existing serialized runtime components unless a compatibility wrapper is provided.

Assembly definitions

- Create separate runtime and editor asmdefs for each releasable lab or Core + all labs if shipping as a single bundle first.
- Editor asmdefs reference runtime asmdefs. Runtime asmdefs never reference editor asmdefs.
- Optional bridges get their own asmdefs and dependencies, e.g. PungentFunk.Utilities.Placement.ColourBridge.
- Third-party dependencies such as TextMeshPro, Input System, Graph Toolkit, or accessibility-facing packages should be declared, optionalized, or isolated behind reflection/bridge code.
- Use assembly definitions to improve compile times and express modularity, not just to silence warnings.

Status labels and documentation

- Use Stable, Experimental, In Progress, Deprecated, and Project Adapter consistently in descriptors, badges, documentation, and lab hubs.
- Each lab needs an overview, setup checklist, core concepts, recipes, troubleshooting, optional integration notes, UI implementation role notes, accessibility notes, package/dependency notes, and API/adapters page.
- Every editor window should have a Help/Notes foldout summarising what the tool changes and what it never changes.
- Every destructive or batch workflow should document its Undo support and limits.
- Major tools should document why their primary presentation is a lab window, inspector integration, TreeView, multi-column view, Scene overlay, context action, graph/canvas tool, Search provider, UI Toolkit view, IMGUI view, or hybrid.

15. Utility Definition of Done

- Registered in PungentUtilityRegistry or a consistent per-lab registrar with complete metadata.
- Primary menu path and optional context menus are cleanly organised.
- Implementation role is deliberately chosen and documented for major tools: EditorWindow, CustomEditor, Focused Inspector, PropertyDrawer, TreeView, MultiColumnView, Search provider, Shortcut command, Scene GUI, overlay, EditorTool, context menu, runtime service, bridge, graph/canvas tool, or hybrid.
- Uses UtilityWindowTheme and UtilityWindowPrefs for visual style and persistent panel state, or maps UI Toolkit styling to the same design tokens.
- Follows Unity-standard component semantics and interaction patterns unless a deliberate, documented alternative is needed.
- Responsive at small window sizes; no avoidable horizontal scrollbar; draggable splitters where panels compete for space.
- All fields have tooltips; enum choices have contextual descriptions; complex operations have inline help.
- Empty states are useful and action-oriented; error/warning/info messages state the problem, cause, and next step where relevant.
- Accessibility expectations are met for contrast, non-colour indicators, focus visibility, readable text, keyboard reachability where feasible, and event-driven accessibility updates for runtime UI where relevant.
- Repeated UI patterns use shared PungentFunk widgets/helpers rather than one-off copies.
- Dense hierarchical or audit-style data uses TreeView, multi-column views, Search tables, or an equivalent stateful list/table pattern where appropriate.
- Scene editing tools separate Scene GUI/Handles from scan/apply logic and keep Scene GUI lightweight.
- Graph/canvas tools are used only where relationships, branching, dependencies, pipelines, or spatial organisation justify them.
- Custom inspectors use serialized property workflows and support multi-object editing where relevant, or clearly explain why not.
- Runtime code compiles without UnityEditor; editor code lives in Editor folders or editor asmdefs.
- No compile-time dependency on game-specific classes.
- Optional integrations degrade gracefully when missing.
- AssetDatabase operations are explicit, safe, batched when appropriate, and do not create import/refresh churn unnecessarily.
- Tools tolerate domain reload, code reload, window close/reopen, and saved editor layouts.

- Scan-heavy tools cache results and refresh explicitly.
- Batch/destructive operations provide preview/dry-run and Undo, or a clear warning where Undo is impossible.
- Tested in a clean project, with no optional labs installed, and with the full bundle installed.

16. Reference Basis for Unity Editor and Foundations Doctrine

This section records the stable documentation categories that influenced the doctrine. It is intentionally concise and not a tutorial or API reference.

- Offline Unity Manual archive audit: approximately 31,170 extracted files including 2,079 Manual HTML pages. Relevant categories included editor extension, UI Toolkit, ImGui, custom windows, custom inspectors, property drawers, TreeView, ListView, Scene View, Gizmos, Handles, overlays, EditorTools, Search, shortcuts, AssetDatabase, package management, assembly definitions, prefabs, domain/code reload, and workspace customization.
- Unity Editor implementation documentation: ImGui, UI Toolkit, UXML, USS, custom editor windows, custom inspectors, property drawers, TreeView, ListView, Scene GUI, Handles, editor overlays, EditorTools, workspace customization, shortcuts, Search, and Graph Toolkit concepts.
- Unity Foundations fundamentals: accessibility, colour palette, iconography, interactions, typography, UI writing, and UX essentials.
- Unity Foundations patterns: authoring flows, content organisation, empty states, errors and messaging, overlays, and windowing.
- Unity Foundations components: button, colour field, dropdown, foldout, help box, label, list view, multi-column views, numeric field, object field, progress bar, radio button, scroll view, search field, slider, tab, text field, toggle, toolbar, tooltip, and tree view.
- Unity package and project structure documentation: Package Manager, custom packages, asset packages, assembly definitions, special folder names, AssetDatabase workflow, asset import/refresh, and prefab workflows.
- Unity accessibility documentation: accessibility hierarchies, accessibility nodes, labels/roles/values/states, screen-reader notifications, hierarchy flattening, and performance-aware updates.

17. Closing Doctrine

PungentFunk Utilities should remain modular in code, cohesive in user experience, conservative in dependencies, accessible in presentation, and deliberate in Unity Editor implementation choices. The suite can feel interconnected without becoming tightly coupled. Core provides discovery and shared UX. Labs provide focused capability. Bridges provide optional collaboration. Project adapters preserve game-specific workflows without contaminating the generic assets.

When in doubt, choose the smaller generic utility, the thinner adapter, the safer apply flow, the clearer layout, the more explicit dependency boundary, the more accessible communication pattern, and the Unity Editor integration point that appears where the user naturally needs the tool.